

Better Method: PrefixSpan

- ▶ Generate all frequent sequences without candidate generation and testing.
- ▶ J. Pei et al. PrefixSpan: Mining sequential patterns efficiently by prefix-projected pattern growth. In Proc. of ICDE'01.
- ▶ Will talk about it next.

PrefixSpan

- ▶ Generate all frequent sequences without candidate generation and testing.
- ▶ Strategy
 - ▶ Divide and conquer
 - ▶ Divide the patterns to be mined into subsets and find patterns in each subset recursively.
 - ▶ Projection-based
 - ▶ Recursively project a sequence database into a set of smaller databases based on the frequent “prefix” mined so far
 - ▶ Mine each projected database to find frequent “suffixes”

Prefix and Suffix

▶ Suppose

- ▶ all the items in an element (i.e. an itemset) of a sequence are listed alphabetically.
- ▶ () is used to enclose an itemset
- ▶ () is omitted for itemsets with a single item

▶ *Prefixes* of sequence $\langle a(abc)(ac)d(cf) \rangle$ are $\langle a \rangle$, $\langle aa \rangle$, $\langle a(ab) \rangle$, $\langle a(abc) \rangle$, $\langle a(abc)a \rangle$,

▶ *Suffixes* of sequence $\langle a(abc)(ac)d(cf) \rangle$:

- ▶ $\langle (abc)(ac)d(cf) \rangle$ is the suffix wrt prefix $\langle a \rangle$
- ▶ $\langle (_bc)(ac)d(cf) \rangle$ is the suffix wrt prefix $\langle aa \rangle$
- ▶ $\langle (_c)(ac)d(cf) \rangle$ is the suffix wrt prefix $\langle a(ab) \rangle$
- ▶ ...

Mining Sequential Patterns by Prefix Projections

- ▶ Step 1: find length-1 sequential patterns
 - ▶ $\langle a \rangle$, $\langle b \rangle$, $\langle c \rangle$, $\langle d \rangle$, $\langle e \rangle$, $\langle f \rangle$
- ▶ Step 2: divide search space. The complete set of freq. seq. can be partitioned into 6 subsets:
 - ▶ The ones having prefix $\langle a \rangle$;
 - ▶ The ones having prefix $\langle b \rangle$;
 - ▶ ...
 - ▶ The ones having prefix $\langle f \rangle$

SID	sequence
10	$\langle a(abc)(ac)d(cf) \rangle$
20	$\langle (ad)c(bc)(ae) \rangle$
30	$\langle (ef)(ab)(df)cb \rangle$
40	$\langle eg(af)cbc \rangle$

min_sup count=2

Finding Freq. Seq. with Prefix <a>

- ▶ Only need to consider sequences in D containing <a>
- ▶ In a sequence containing <a>, only the subsequence (suffixes) prefixed with the first occurrence of <a> should be considered.
- ▶ The collection of such subsequences is called *<a>-projected database*:

<(abc)(ac)d(cf)>,
<(_d)c(bc)(ae)>,
<(_b)(df)cb>,
<(_f)cbc>.

D

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

min_sup count=2

Next: recursively mine *<a>-projected database* to find frequent sequences in the projected database.

Finding Freq. Seq. with Prefix <a> (Cont'd)

- ▶ Find local frequent length-1 sequences by scanning <a>-projected database
 - ▶ *<a>-projected database*: <(abc)(ac)d(cf)>, <(_d)c(bc)(ae)>, <(_b)(df)cb>, <(_f)cbc> min_sup count=2
 - ▶ *Local frequent length-1 sequences*:
<a>:2, :4, <(_b)>:2, <c>:4, <d>:2, <f>:2
- ▶ Generate all the length-2 freq. seq. having prefix <a>:
<aa>:2, <ab>:4, <(ab)>:2, <ac>:4, <ad>:2, <af>:2
- ▶ Further partition frequent sequences with prefix <a> into 6 subsets
 - ▶ Having prefix <aa>;
 - ▶ Having prefix <ab>;
 - ▶ ...
 - ▶ Having prefix <af>

Finding Freq. Seq. with Prefix <aa>

- ▶ From <a>-projected database:
<(abc)(ac)d(cf)>, <(_d)c(bc)(ae)>, <(_b)(df)cb>, <(_f)cbc>
- ▶ Generate <aa>-projected database:
<(_bc)(ac)d(cf)>, <(_e)>
- ▶ No local frequent items, stop growing prefix <aa>.

min_sup count=2

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Finding Freq. Seq. with Prefix <ab>

- ▶ From <a>-projected database:
<(abc)(ac)d(cf)>, <(_d)c(bc)(ae)>, <(_b)(df)cb>, <(_f)cbc>
- ▶ Generate <ab>-projected database:
<(_c)(ac)d(cf)>, <(_c)(ae)>, <c>
- ▶ Local frequent length-1 sequences
 - ▶ <a>:2, <c>:2, <(_c)>:2
- ▶ Generate length-3 freq. seq. with prefix <ab>:
 - ▶ <aba>:2, <abc>:2, <a(bc)>:2
- ▶ Further partition the set of freq. Seq. prefixed with <ab>:
 - ▶ Sequences with prefix <aba>
 - ▶ Sequences with prefix <abc>
 - ▶ Sequences with prefix <a(bc)>

Finding Freq. Seq. with Prefix <aba>

- ▶ From <ab>-projected database:
 <(_c) (ac)d(cf)>, <(_c)(ae)>, <c>
- ▶ Generate <aba>-projected database:
 - ▶ <(_c)d(cf)>, <(_e)>
- ▶ No local frequent length-1 sequence, stop growing <aba>.

Finding Freq. Seq. with Prefix <abc>

- ▶ From <ab>-projected database:
 <(_c) (ac)d(cf)>, <(_c)(ae)>, <c>
- ▶ Generate <abc>-projected database:
 - ▶ <d(cf)>
- ▶ No local frequent length-1 sequence, stop growing <abc>.

Finding Freq. Seq. with Prefix $\langle a(bc) \rangle$

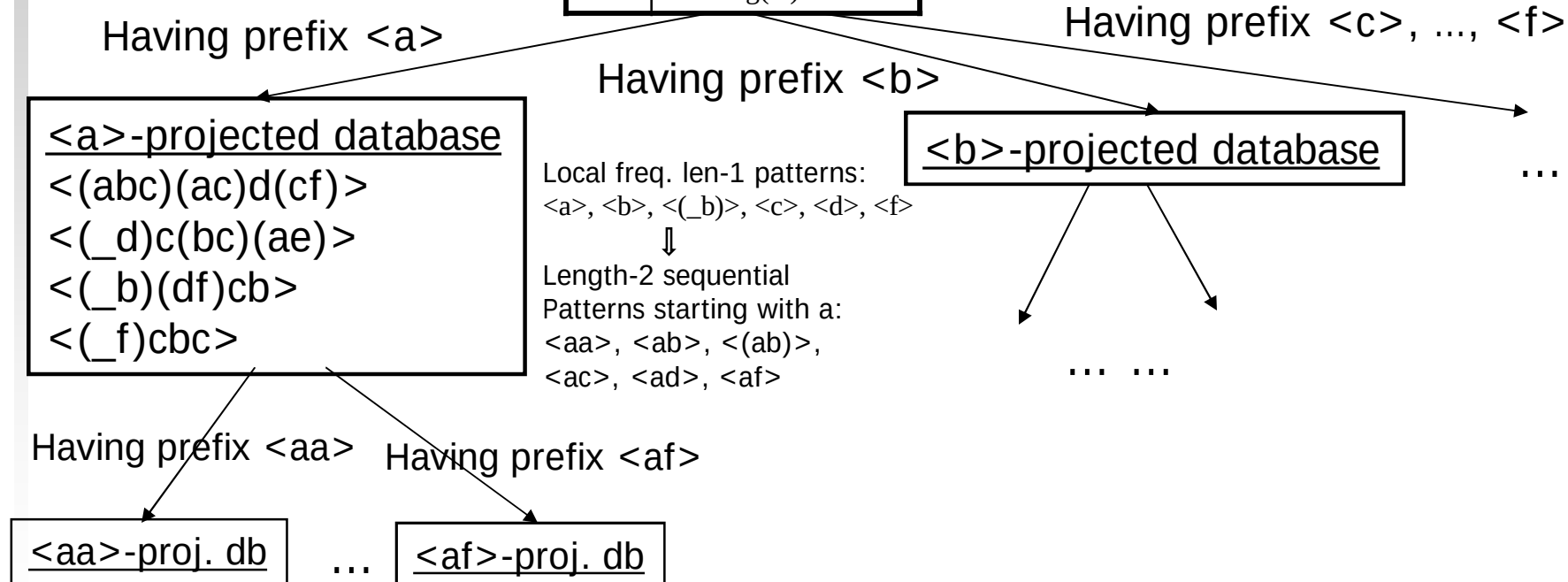
- ▶ From $\langle ab \rangle$ -projected database:
 - $\langle (_c) (ac)d(cf) \rangle$, $\langle (_c)(ae) \rangle$, $\langle c \rangle$
- ▶ Generate $\langle a(bc) \rangle$ -projected database:
 - ▶ $\langle (ac)d(cf) \rangle$, $\langle (ae) \rangle$
- ▶ Local frequent length-1 sequences:
 - ▶ $\langle a \rangle:2$
- ▶ Generate length-4 freq. seq. with prefix $\langle a(bc) \rangle$:
 - ▶ $\langle a(bc)a \rangle:2$
- ▶ Further mining $\langle a(bc)a \rangle$ -projected database returns no frequent sequence prefixed with $\langle a(bc)a \rangle$.
- ▶ This finishes generating all the patterns prefixed with $\langle ab \rangle$.

Completeness of PrefixSpan

SDB

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Length-1 sequential patterns
<a>, , <c>, <d>, <e>, <f>



Efficiency of PrefixSpan

- ▶ No candidate sequence needs to be generated
- ▶ Projected databases keep shrinking
- ▶ Major cost of PrefixSpan:
 - ▶ constructing projected databases.
 - ▶ overlapping suffixes in recursively projected databases
 - ▶ can be improved by *Pseudo-projections*

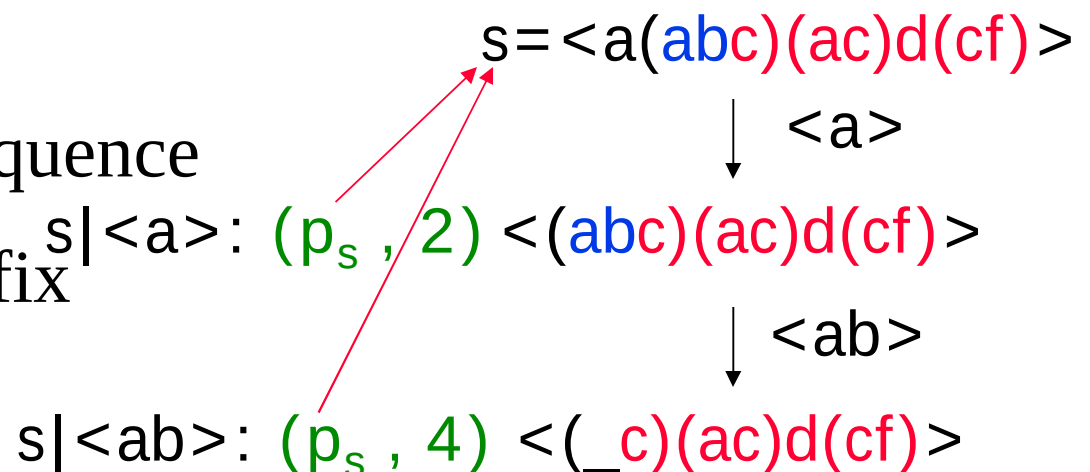
Speed-up by Pseudo-projection

- ▶ Major cost of PrefixSpan: projection
 - ▶ Suffixes of a sequence often appear repeatedly in recursively projected databases
 - ▶ $\langle (abc)(ac)d(cf) \rangle$ appears in $\langle a \rangle$ -projected database
 - ▶ $\langle (_c)(ac)d(cf) \rangle$ appears in $\langle ab \rangle$ -projected database

- ▶ When (projected) database can be held in main memory, **use pointers (p, o) to form projections**

- ▶ **p**: Pointer to the sequence

- ▶ **o**: Offset of the suffix



Pseudo-Projection vs. Physical Projection

- ▶ Pseudo-projection avoids physically copying suffixes
 - ▶ Efficient in running time and space when database can be held in main memory
- ▶ However, it is not efficient when database cannot fit in main memory
 - ▶ Disk-based random accessing is very costly
- ▶ Suggested Approach:
 - ▶ Integration of physical and pseudo-projection
 - ▶ Swapping to pseudo-projection when the projected database fits in memory

Experiments and Performance Analysis

- ▶ Comparing PrefixSpan with GSP, FreeSpan and SPADE in large databases
 - ▶ GSP (IBM Almaden, Srikant & Agrawal EDBT'96)
 - ▶ FreeSpan (J. Han J. Pei, B. Mortazavi-Asi, Q. Chen, U. Dayal, M.C. Hsu, KDD'00)
 - ▶ SPADE (Zaki, 01)
- ▶ PrefixSpan is fastest and scalable.

Problem of Sequential Pattern Mining

- ▶ GSP and PrefixSpan finds all the sequences that satisfy the support threshold.
- ▶ A long frequent sequence contains a combinatorial number of frequent subsequences:
 - ▶ For a length-100 sequential pattern, there exist $2^{100} - 1$ nonempty subsequences.
- ▶ Problem: too many patterns are generated.

Solutions

- ▶ Mining maximal or closed sequential patterns
 - ▶ Maximal sequential pattern:
 - ▶ A frequent sequence s is *maximal* if there exists no frequent super-sequence of s .
 - ▶ Closed sequential pattern:
 - ▶ A sequence s is *closed* if there exists no super-sequence of s with the same support as s .
 - ▶ *CloSpan* (Yan, Han and Afshar, 2003): an efficient closed sequential pattern mining method.

Solutions (Cont'd)

- ▶ Constraint-based mining of sequential patterns
 - ▶ Constraints
 - ▶ Item constraint
 - ▶ Find patterns containing a, b, c, but no d.
 - ▶ Length constraint
 - ▶ Find patterns having at least (or at most) 10 items
 - ▶ Super-pattern constraint
 - ▶ Find patterns that contain <(PC)(Digital-camera)>
 - ▶ Aggregate constraint
 - ▶ Find patterns the average price of whose items is over \$100.

Solutions (Cont'd)

- ▶ More constraints
 - ▶ Regular expression constraint
 - ▶ $\langle a^*\{bb|(bc)d|dd\} \rangle$
 - ▶ Duration constraint
 - ▶ Find patterns of events about +/- 24 hours of a shooting
 - ▶ Gap constraint
 - ▶ Find purchasing patterns such that the gap between each consecutive purchases is less than 1 month.

Solutions (Cont'd)

▶ Properties of constraints

▶ Anti-monotonic constraint

- ▶ If a sequence s satisfies constraint C , so does every non-empty subsequence of s
- ▶ Examples: $\text{support}(s) \geq 5\%$, $\text{length}(s) < 10$

▶ Monotonic constraint

- ▶ If a sequence s satisfies constraint C , so does every super sequence of s .
- ▶ Examples: $\text{length}(s) \geq 10$, super pattern constraints.

▶ A systematic study on constraint-based sequential pattern mining:

- ▶ J. Pei, J. Han, and W. Wang. "Mining Sequential Patterns with Constraints in Large Databases". *In Proceedings of CIKM'02.*

Other Research on Frequent Pattern Mining

- ▶ Mining other kinds of frequent patterns
 - ▶ Frequent tree/graph mining
 - ▶ Find common structural components
 - ▶ Examples of applications
 - ▶ XML documents can be modeled as trees
 - ▶ Molecule or biochemical structures can be modeled as graphs
 - ▶ Web connection structures can be modeled as graphs
- ▶ Finding frequent patterns from data streams
 - ▶ Only one scan of DB is allowed.

Other Research on Frequent Pattern Mining (Cont'd)

- ▶ Finding high-utility patterns (either itemsets or sequences)
 - ▶ Consider the quantity of the item inside a transaction.
 - ▶ Consider the importance (e.g., price) of an item
 - ▶ Find patterns whose revenue is at least, e.g., \$1000

Frequent Pattern Mining Resources

- ▶ Web site: <http://fimi.cs.helsinki.fi/> contains some frequent itemset mining implementations, datasets and papers.
- ▶ SPMF: open-source data mining library containing frequent sequence and itemset mining:
<http://www.philippe-fournier-viger.com/spmf/>